

E2 — Arquitectura modular, Clean Code y SOLID

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: convertir el script de la E1, que era un único fichero que se leía de arriba abajo, en un **paquete de Python** dividido en módulos, cada uno con una responsabilidad. El programa se ejecuta desde la línea de comandos con argumentos.

Estructura creada dentro de `1_programacion_avanzada_python/`:

```
ventas_app/
__init__.py    # convierte la carpeta en paquete
loader.py      # carga de datos (CSV o JSON) + DataLoadError
validator.py    # validación de filas + ValidationError
metrics.py     # agregaciones y resumen de calidad
cli.py         # punto de entrada: argparse + orquestación
DESIGN.md      # principios SOLID y Clean Code aplicados
```

Ejecución (desde `1_programacion_avanzada_python/`):

```
python -m ventas_app.cli --input Datos/ventas.csv --output Datos/ventas_limpias.csv
```

Flujo del programa (lo que hace `cli.py`):

Paso	Módulo	Qué ocurre
1. Leer argumentos	<code>cli.py</code>	<code>argparse</code> obtiene las rutas de <code>--input</code> y <code>--output</code>
2. Cargar	<code>loader.py</code>	<code>load(path)</code> lee el CSV o JSON según la extensión y devuelve un <code>DataFrame</code> , o lanza <code>DataLoadError</code>
3. Validar	<code>validator.py</code>	<code>validar_ventas()</code> separa válidas e inválidas o lanza <code>ValidationError</code>
4. Métricas	<code>metrics.py</code>	importe por región, top 3 productos, clientes recurrentes
5. Exportar	<code>cli.py</code>	guarda <code>ventas_limpias.csv</code> y <code>calidad_datos.json</code>

Resultados: son idénticos a los de la E1 (150 filas, 140 válidas, 10 inválidas e importe total de 96 598,17). Esto confirma que la reorganización **no ha cambiado el comportamiento**, que es justo lo que se busca al refactorizar.

Qué se ha añadido respecto a la E1:

- Excepciones propias con mensajes claros.
- Comprobación de que existen las columnas obligatorias.
- `logging` en lugar de `print`.
- Constantes con nombre en lugar de números sueltos.
- Lectura de ficheros JSON dentro de `load()` (versión sencilla de la extensión del enunciado).

2. Conceptos teóricos

2.1 Módulos y paquetes

- **Módulo:** cualquier fichero `.py`. Se importa por su nombre sin la extensión (`import loader`).
- **Paquete:** una carpeta que agrupa módulos. El fichero `__init__.py` le indica a Python que esa carpeta es un paquete. Puede estar vacío o contener solo una descripción.
- **Importar desde un paquete:**

```
from ventas_app import loader, metrics, validator
loader.load("Datos/ventas.csv") # se llama a la función a través del módulo
```

- **Ventaja de dividir en módulos:** cada fichero es corto y se entiende solo; los cambios quedan localizados, y las piezas se pueden probar y reutilizar por separado.

2.2 `python -m` y `if __name__ == "__main__"`

- `python -m ventas_app.cli` ejecuta el módulo `cli` **como parte del paquete**. Así funcionan los imports del tipo `from ventas_app import ...`.
- Hay que lanzarlo desde la carpeta que **contiene** `ventas_app/`, porque Python busca el paquete en la carpeta actual.
- `__name__` es una variable que Python crea en cada módulo:
 - Vale `"__main__"` si el fichero se está ejecutando directamente.
 - Vale el nombre del módulo (`"ventas_app.cli"`) si se importa desde otro fichero.

```
if __name__ == "__main__":
    main()
```

Esto hace que `main()` solo se ejecute al lanzar el programa, y no al importarlo (por ejemplo, desde los tests).

2.3 Excepciones propias

```
class DataLoadError(Exception):
    """Error que se lanza cuando no se pueden cargar los datos."""
```

- Una excepción propia es una **clase que hereda de `Exception`**. No necesita código dentro, basta con el docstring.
- **Lanzar** un error: `raise DataLoadError("No existe el fichero: ...")`.
- **Capturar** un error:

```
try:
    ventas = loader.load(args.input)
except (loader.DataLoadError, validator.ValidationError) as error:
    logger.error(error)
    sys.exit(1)
```

- **Por qué crear errores propios:** el mensaje explica el problema en términos del negocio ("falta la columna X") en vez de una traza larga de pandas. Además, quien llama a la función puede capturar solo esos errores concretos.
- `sys.exit(1)` termina el programa con **código de salida 1**, que indica que ha habido un error. El código `0` significa que todo ha ido bien. Scripts y herramientas de CI usan este código para saber si el programa ha fallado.

2.4 `argparse`: argumentos por línea de comandos

```
parser = argparse.ArgumentParser(description="Limpieza y métricas de ventas")
parser.add_argument("--input", required=True, help="Ruta del fichero de ventas")
parser.add_argument("--output", required=True, help="Ruta del CSV de ventas limpias")
args = parser.parse_args()
args.input  # valor escrito después de --input
```

- Permite ejecutar el mismo programa con distintos ficheros **sin tocar el código**.
- `required=True` hace que el argumento sea obligatorio; si falta, `argparse` muestra un error.
- `help=` es el texto que aparece con `python -m ventas_app.cli --help`, que se genera automáticamente.

2.5 `logging` en lugar de `print`

```
logging.basicConfig(level=logging.INFO, format="%(levelname)s - %(message)s")
logger = logging.getLogger(__name__)
logger.info("Filas cargadas: %d", len(ventas))
logger.error("No existe el fichero")
```

Nivel	Uso
DEBUG	Detalles para depurar
INFO	Funcionamiento normal
WARNING	Algo raro, pero el programa sigue
ERROR	Algo ha fallado
CRITICAL	Fallo grave

- **Ventajas frente a `print`** : cada mensaje indica su nivel, se puede filtrar (por ejemplo, mostrar solo los errores), se puede enviar a un fichero y el formato se configura en un solo sitio.
- `level=logging.INFO` muestra los mensajes de nivel INFO o superior.
- `%s` y `%d` son marcadores que logging rellena con los valores que van después (`%s` para texto, `%d` para enteros).

2.6 Nuevas herramientas de `pathlib`

Uso	Resultado para <code>Path("Datos/ventas.csv")</code>
<code>ruta.exists()</code>	True / False según exista el fichero
<code>ruta.suffix</code>	".csv"
<code>ruta.with_name("calidad_datos.json")</code>	Datos/calidad_datos.json (misma carpeta, otro nombre)

2.7 SOLID

Son cinco principios de diseño que hacen que el código sea fácil de mantener y ampliar.

Letra	Principio	Idea en una frase
S	Single Responsibility (Responsabilidad única)	Cada módulo o clase tiene un solo motivo para cambiar
O	Open/Closed (Abierto/cerrado)	Abierto a extender , cerrado a modificar lo que ya funciona
L	Liskov Substitution (Sustitución de Liskov)	Una clase hija debe poder usarse donde se usa la clase padre sin romper nada
I	Interface Segregation (Segregación de interfaces)	Mejor varias interfaces pequeñas que una grande con métodos que no todos usan
D	Dependency Inversion (Inversión de dependencias)	Depender de abstracciones , no de implementaciones concretas

Los tres aplicados en la práctica:

- **SRP**: `loader` solo lee, `validator` solo valida, `metrics` solo calcula y `cli` solo orquesta. Por ejemplo, si cambian las reglas de validación, solo se toca `validator.py`.

- **OCP:** en `validator.py`, la lista `COLUMNAS_POSITIVAS` se recorre con un bucle. Para exigir que otra columna sea positiva basta con añadirla a la lista, sin modificar la función.
- **DIP:** `validator.py` y `metrics.py` reciben un `DataFrame` y no saben de dónde vienen los datos. Por eso añadir la lectura de JSON solo ha requerido cambiar `loader.py`.

Alternativa: DIP con Protocol (solo teoría, no implementada)

Es la versión que plantea el enunciado para la extensión (`JsonSalesRepository`). En la práctica se ha optado por la versión sencilla (un `if` según la extensión dentro de `load()`), porque para un único dataset ofrece el mismo resultado con menos código. Aun así, conviene conocerla para el examen.

Un **Protocol** define una "interfaz": qué métodos debe tener un objeto, sin decir cómo se implementan. Cualquier clase que tenga esos métodos la cumple **sin necesidad de heredar** de ella. Esto se llama *duck typing*: "si camina como un pato y hace cuac, es un pato".

```
class SalesRepository(Protocol):          # la abstracción
    def load(self) -> pd.DataFrame: ...

class CsvSalesRepository:                 # implementación concreta 1
    def __init__(self, path): self.path = path
    def load(self) -> pd.DataFrame: return pd.read_csv(self.path)

class JsonSalesRepository:                # implementación concreta 2 (extensión)
    def __init__(self, path): self.path = path
    def load(self) -> pd.DataFrame: return pd.read_json(self.path)

REPOSITARIOS = {".csv": CsvSalesRepository, ".json": JsonSalesRepository}

def crear_repositorio(path) -> SalesRepository:
    ruta = Path(path)
    ...                                     # comprueba existencia y extensión
    return REPOSITARIOS[ruta.suffix](ruta)

def load(path) -> pd.DataFrame:
    repositorio = crear_repositorio(path) # no importa si es CSV o JSON
    return repositorio.load()             # solo se usa el método de la interfaz
```

Conceptos de clases que aparecen:

- `class Nombre:` define una clase, que es una "plantilla" para crear objetos.
- `__init__` es el constructor: se ejecuta al crear el objeto y guarda sus datos. Por ejemplo, `CsvSalesRepository(ruta)` crea un objeto y guarda la ruta en `self.path`.
- `self` es el propio objeto. `self.path` es un **atributo** (un dato del objeto).
- `def load(self)` es un **método**, es decir, una función que pertenece a la clase.

- El diccionario `REPOSITARIOS` guarda **clases** (no objetos), y `REPOSITARIOS[".csv"]` (ruta) crea el objeto. A esta función que decide qué clase instanciar se le suele llamar *factoría*.

Por qué cumple DIP y OCP a la vez: para admitir un formato nuevo (por ejemplo Excel), basta con crear `ExcelSalesRepository` con su método `load()` y añadirlo a `REPOSITARIOS`. No hay que modificar `load()`, `validator.py`, `metrics.py` ni `cli.py`. En la versión sencilla, en cambio, habría que añadir otro `if` dentro de `load()`: es más simple, pero modifica una función que ya existe.

2.8 Clean Code

Mala práctica	Cómo se ha corregido
Nombres crípticos (<code>df2</code> , <code>res</code> , <code>x</code>)	Nombres que describen qué contienen o hacen: <code>importe_por_region</code> , <code>clientes_recurrentes</code>
Números mágicos (<code>> 0</code> , <code>.head(3)</code>)	Constantes con nombre en MAYÚSCULAS: <code>VALOR_MINIMO</code> , <code>NUM_TOP_PRODUCTOS</code>
<code>print</code> de depuración sueltos	<code>logging</code> con niveles
Funciones largas que hacen de todo	Funciones cortas con una sola tarea
Errores genéricos	Excepciones propias con mensajes claros

- **Número mágico:** un valor literal en el código cuyo significado no es obvio. Si se le da nombre como constante, se entiende al leerlo y, si cambia, se cambia en un solo sitio.
- **Convención:** las constantes se escriben en MAYÚSCULAS al principio del módulo.

2.9 Refactorizar

- **Refactorizar** es cambiar la estructura del código **sin cambiar lo que hace**.
- Forma de comprobarlo: el resultado final tiene que ser el mismo antes y después del cambio. En esta práctica, los mismos 150/140/10 y el mismo importe total que en la E1.

3. Comandos usados

```
cd 1_programacion_avanzada_python
mkdir ventas_app
touch ventas_app/__init__.py ventas_app/loader.py ventas_app/validator.py \
    ventas_app/metrics.py ventas_app/cli.py ventas_app/DESIGN.md
python -m ventas_app.cli --input Datos/ventas.csv --output Datos/ventas_limpias.csv
python -m ventas_app.cli --help      # ver la ayuda generada por argparse
```

- `mkdir` crea una carpeta; `touch` crea ficheros vacíos.

- Error típico: `AttributeError: module 'X' has no attribute 'Y'`. Significa que el módulo no contiene esa función, normalmente porque el fichero no se guardó o el nombre está mal escrito.

4. Preguntas típicas de examen (E2)

1. **¿Qué diferencia hay entre un módulo y un paquete?** Un módulo es un fichero `.py`; un paquete es una carpeta con varios módulos y un `__init__.py`.
2. **¿Para qué sirve `__init__.py`?** Para indicar a Python que la carpeta es un paquete importable.
3. **¿Para qué sirve `if __name__ == "__main__"`?** Para que el código solo se ejecute cuando se lanza el fichero directamente, y no cuando se importa.
4. **¿Cómo se crea una excepción propia?** Con una clase que hereda de `Exception`:

```
class DataLoadError(Exception): ...
```
5. **¿Qué diferencia hay entre `raise` y `try/except`?** `raise` lanza un error; `try/except` lo captura y decide qué hacer con él.
6. **¿Qué significa `sys.exit(1)`?** Que el programa termina indicando que ha habido un error. `0` indica éxito.
7. **¿Por qué usar `logging` en vez de `print`?** Porque tiene niveles, se puede filtrar y redirigir, y el formato se configura en un solo sitio.
8. **¿Qué ventaja tiene `argparse`?** Permite cambiar los datos de entrada y salida sin tocar el código, y genera la ayuda (`--help`) automáticamente.
9. **Explica SRP con un ejemplo de la práctica.** `validator.py` solo valida; si cambian las reglas de validación, no hay que tocar ni `loader` ni `metrics`.
10. **Explica OCP con un ejemplo.** Añadir una nueva columna a `COLUMNAS_POSITIVAS` amplía la validación sin modificar la función `validar_ventas`.
11. **Explica DIP con un ejemplo.** `metrics.py` y `validator.py` dependen de un `DataFrame` y no de un CSV concreto; por eso añadir la lectura de JSON solo afectó a `loader.py`. En su versión formal, se usa un `Protocol` (`SalesRepository`) del que dependen las clases de alto nivel.
12. **¿Qué es un número mágico y cómo se corrige?** Un valor literal sin explicación; se corrige sustituyéndolo por una constante con nombre.
13. **¿Qué es refactorizar y cómo se comprueba que se ha hecho bien?** Cambiar la estructura sin cambiar el comportamiento; se comprueba obteniendo los mismos resultados que antes.
14. **¿Qué es un `Protocol`?** Una interfaz que define qué métodos debe tener un objeto. Cualquier clase con esos métodos la cumple sin heredar de ella (*duck typing*).
15. **¿Cómo añadirías un nuevo formato de datos (por ejemplo Excel)?** En la versión sencilla, con otro `if ruta.suffix == ".xlsx"` dentro de `load()`. En la versión con `Protocol`, creando `ExcelSalesRepository` con un método `load()` y registrándolo en `REPOSITARIOS`. En ambos casos, `validator.py`, `metrics.py` y `cli.py` no cambian.
16. **¿Qué son `__init__` y `self`?** `__init__` es el constructor, que se ejecuta al crear el objeto; `self` es el propio objeto, y permite guardar y leer sus atributos (`self.path`).